Flask Lezione 3

Introduzione

Nelle scorse lezioni abbiamo analizzato i fondamenti di flask, concentrandoci sugli elementi essenziali per la creazione di un'applicazione web.
In particolare abbiamo visto:

- Come istanziare l'app Flask,
- Come definire route statici e route dinamiche,
- L' utilizzo della funzione `url_for()` per la ricostruzione degli URL
- L'uso dei decorators in Flask
- e infine come flask possa essere utilizzato per implementare **API RESTful**, seguendo i principi dell'architettura REST.
  Queste basi ci permettono ora di comprendere meglio la struttura e la filosofia del framework.

Flask – Il Microframework Python

Flask è un **framework web Python minimale e flessibile**, progettato per semplificare lo sviluppo di applicazioni web e di **API**.

Il termine **"microframework"** non indica che Flask sia limitato nelle funzionalità, **ma che fornisce solo il nucleo essenziale, lasciando allo sviluppatore la libertà di estendere il framework tramite librerie ed estensioni esterne, in base alle esigenze del progetto.**

Filosofia di Flask

La filosofia di Flask si basa su alcuni principi chiave:

- fornisce **il core minimo indispensabile**;
- tutte le funzionalità aggiuntive sono **estendibili tramite plugin**;
- non impone una struttura rigida al progetto;
- garantisce **massima libertà progettuale** allo sviluppatore.

Grazie a queste caratteristiche, Flask risulta particolarmente adatto per:

- lo sviluppo di **API REST**;
- la realizzazione di **microservizi**;

- applicazioni **piccole e medie**;
- progetti didattici, prototipi e proof-of-concept.

Creazione dell'ambiente virtuale

Abbiamo visto come installare ed eseguire Flask sulla nostra macchina, ma nella pratica è fortemente consigliato **isolare le dipendenze del progetto**.
Questo si ottiene utilizzando un **ambiente virtuale (virtual environment)**, che permette di evitare conflitti tra librerie di progetti diversi.

Per creare un ambiente virtuale in Python, da terminale si utilizza il seguente comando:

Shell

```
python -m venv venv
```

Una volta creato, l'ambiente virtuale deve essere attivato.
La procedura varia in base al sistema operativo:

1. **Linux/macOS**

Shell

```
source venv/bin/activate
```

2. **Windows**

Shell

```
venv\Scripts\activate
```

Dopo l'attivazione, tutte le librerie installate tramite `pip` verranno associate esclusivamente a questo progetto.

Prima applicazione in Flask

Dopo aver creato e attivato l'ambiente virtuale, possiamo iniziare a scrivere la nostra **prima applicazione Flask**.

Come già visto nella lezione Introduzione a Flask nel capitolo Scrivere la prima applicazione in Flask
Flask permette di creare rapidamente un **server web minimale**, definire **route**

(cioè gli URL a cui il server risponde) e gestire **richieste e risposte**. Questo include sia contenuti statici, come una stringa semplice, sia risposte più complesse, come dati in formato JSON per le API REST.

Ecco un esempio di applicazione base:

Python

```python
from flask import Flask

# 1. Creare un'istanza dell'app Flask
app = Flask(__name__)

# 2. Definire una route statica
@app.route('/')
def home():
    # Quando un client visita l'URL '/', questa funzione viene eseguita
    # e restituisce una semplice stringa come risposta.
    return 'Ciao da Flask!'

# 3. Definire una route per lo stato dell'API
@app.route('/api/status')
def status():
    # Questa route restituisce un dizionario Python,
    # che Flask converte automaticamente in JSON.
    return {'status': 'OK', 'message': 'API funzionante'}

# 4. Avviare il server
if __name__ == '__main__':
    # Il server parte in modalità debug, utile per sviluppo:
    # aggiorna automaticamente il server ad ogni modifica del codice
    # e mostra errori dettagliati.
    app.run(debug=True)

```

Spiegazione del codice:

1. **Creazione dell'istanza Flask**
   - `app = Flask(__name__)` **crea un'istanza dell'applicazione.**
   - **Questo oggetto gestirà tutte le route, le richieste e le risposte.**
2. **Definizione delle route**

- **Le route vengono definite con il decoratore** `@app.route('URL')`**.**
- **La funzione sottostante viene eseguita quando un client visita quell'URL.**
- Nel nostro esempio:
  - `'/'` **restituisce un semplice messaggio di benvenuto.**
  - `'/api/status'` **restituisce un dizionario, che Flask converte automaticamente in JSON, utile per testare l'API**.

3. **Avvio del server**
   - Il blocco `if __name__ == '__main__':` **assicura che il server parta solo se eseguiamo direttamente questo file.**
   - `app.run(debug=True)` **avvia il server in modalità debug, rendendo più facile sviluppare e testare le modifiche.**

Esecuzione:
Per avviare l'applicazione, nel terminale digitiamo:

Shell

```
python app.py
```

Il server sarà disponibile all'indirizzo:

- `http://127.0.0.1:5000`

Ora, visitando questo URL nel browser o usando strumenti come **Postman**, possiamo vedere la risposta del server.

Routing in Flask

Le **route** definiscono gli endpoint a cui il client può fare richieste.
Un **endpoint** è semplicemente un URL gestito dall'applicazione, associato a una funzione Python che viene eseguita quando quel percorso viene visitato.
Possono essere:

- **statiche:** cioè con un percorso fisso;,
- **dinamiche** (con parametri): cioè con parametri variabili nell'URL;
- **query parameters:** per filtrare o specificare informazioni nella richiesta.
- **Con metodi HTTP multipli**, per distinguere diverse azioni sulla stessa risorsa.
  Vediamo alcuni esempi pratici.

1. **Route statica**

```python
@app.route('/info')
def info():
    return {'app': 'Libreria API', 'versione': '1.0'}
```

Spiegazione:

- `@app.route('/info')` **definisce che, quando un client visita l'URL** `/info`**, viene eseguita la funzione** `info()`**.**
- La funzione restituisce un **dizionario Python**, che Flask converte automaticamente in **JSON**.
- **Questo è utile per fornire informazioni fisse sull'applicazione, come il nome o la versione.**

2. **Route dinamica (con parametri URL)**

```python
@app.route('/libri/<int:libro_id>')
def get_libro(libro_id):
    return {'id': libro_id, 'titolo': f'Libro {libro_id}'}
```

Spiegazione:

- `<int:libro_id>` **indica un parametro dinamico nell'URL.**
- **Flask cattura il valore passato dall'utente e lo passa come argomento alla funzione (** `libro_id` **).**
- **Possiamo quindi generare risposte personalizzate in base al parametro, ad esempio restituendo i dati di un libro specifico**.

3. **Route con query parameters**

```python
@app.route('/ricerca')
def ricerca():
    termine = request.args.get('q', '')
    return {'query': termine, 'risultati': []}
```

Spiegazione:

- **I query parameters sono valori passati nell'URL dopo il `?`, ad esempio: `/ricerca?q=fantasy`.**
- `request.args.get('q', '')` **recupera il valore del parametro `q` dalla richiesta.**
- **In questo esempio, la funzione restituisce un dizionario con il termine di ricerca e un array vuoto di risultati (simulazione).**

4. **Route con metodi HTTP multipli**

Python

```python
@app.route('/libri', methods=['GET', 'POST'])
def gestisci_libri():
    if request.method == 'GET':
        return {'libri': []}
    elif request.method == 'POST':
        return {'messaggio': 'Libro creato'}, 201
```

Spiegazione:

- La stessa route `/libri` può gestire **più metodi HTTP** usando `methods=['GET', 'POST']`.
- `request.method` permette di distinguere il tipo di richiesta:
  - **GET**: **restituisce i dati della risorsa, in questo caso una lista di libri;**
  - **POST**: **crea una nuova risorsa, qui simulata con un messaggio di conferma.**
- Il numero `201` è un **codice di stato HTTP** che indica "risorsa creata con successo".

Gestione dei dati (simulazione database)

In questa sezione vediamo come Flask può gestire i dati di un'applicazione. Per semplicità, **simuliamo un database** creando una **lista di dizionari**, dove ogni dizionario rappresenta un libro con `id`, `titolo` e `autore`.

Python

```python
from flask import Flask, request, jsonify

```

```python
app = Flask(__name__)

libri = [
    {'id': 1, 'titolo': '1984', 'autore': 'George Orwell'},
    {'id': 2, 'titolo': 'Il nome della rosa', 'autore': 'Umberto Eco'}
]
```

- `libri` è una **lista di dizionari Python** che funge da archivio temporaneo.
- In un'applicazione reale, qui verrebbe collegato un vero database, ad esempio con **SQLAlchemy**.

Ottenere i libri (GET)

Python

```python
@app.route('/libri', methods=['GET'])
def get_libri():
    return jsonify({'libri': libri, 'totale': len(libri)})
```

Spiegazione:

- **La route `/libri` è associata al metodo HTTP `GET`, utilizzato per recuperare informazioni.**
- `jsonify()` **converte la lista Python in JSON, che è lo standard per le API REST.**
- **Restituiamo sia la lista dei libri sia un campo `totale` con il numero di libri presenti.**
- In questo modo, un client può facilmente ottenere tutti i libri disponibili nella "collezione".

Aggiungere un nuovo libro (POST)

Python

```python
@app.route('/libri', methods=['POST'])
def add_libro():
    dati = request.get_json()
    if not dati or 'titolo' not in dati:
        return {'errore': 'Titolo obbligatorio'}, 400

    nuovo_libro = {
```

```python
        'id': len(libri) + 1,
        'titolo': dati['titolo'],
        'autore': dati.get('autore', 'Sconosciuto')
    }
    libri.append(nuovo_libro)
    return jsonify(nuovo_libro), 201
```

Spiegazione:

1. `request.get_json()` **legge il corpo della richiesta e lo converte in un dizionario Python.**
2. Controlliamo che sia presente il campo `titolo`; se manca, restituiamo un **errore 400 (Bad Request)**.
3. Creiamo un nuovo dizionario `nuovo_libro` con:
   - `id`: **assegnato automaticamente come lunghezza attuale della lista +1,**
   - `titolo`: **preso dai dati inviati dal client,**
   - `autore`: **opzionale, con valore predefinito** `"Sconosciuto"` **se non fornito.**
4. Aggiungiamo il nuovo libro alla lista `libri`.
5. Restituiamo il libro appena creato in formato JSON, insieme al codice **201 (Created)**, che indica che la risorsa è stata creata con successo

Gestione degli errori

Quando sviluppiamo API REST, è importante **gestire correttamente gli errori**, restituendo al client messaggi chiari e codici di stato HTTP appropriati.
Flask permette di definire **error handler personalizzati:**

- **cioè funzioni che vengono eseguite automaticamente quando si verifica un errore specifico.**

Error handler globali:

Python

```python
@app.errorhandler(404)
def not_found(error):
    return jsonify({'errore': 'Risorsa non trovata', 'codice': 404}), 404

@app.errorhandler(400)
```

```python
def bad_request(error):
    return jsonify({'errore': 'Richiesta non valida', 'codice': 400}), 400

@app.errorhandler(500)
def internal_error(error):
    return jsonify({'errore': 'Errore interno del server', 'codice': 500}), 500
```

Spiegazione:

- `@app.errorhandler(codice)` **associa una funzione all'errore HTTP specificato (** `404` **,** `400` **,** `500` **).**
- All'interno della funzione, possiamo costruire una **risposta JSON** con informazioni utili al client:
  - `errore` : **messaggio descrittivo dell'errore,**
  - `codice` : **codice HTTP dell'errore.**
- L'ultimo parametro della funzione (`404` , `400` , `500`) indica al client il codice di stato della risposta.
- In questo modo, ogni volta che si verifica un errore di quel tipo, Flask invia automaticamente la risposta definita, rendendo le API più **coerenti e leggibili**.

Gestione degli errori direttamente in una route

Oltre agli error handler globali, possiamo gestire gli errori anche **all'interno di una route specifica**, per intercettare situazioni particolari:

Python

```python
@app.route('/libri/<int:libro_id>')
def get_libro(libro_id):
    libro = next((l for l in libri if l['id'] == libro_id), None)
    if not libro:
        return jsonify({'errore': f'Libro {libro_id} non trovato'}), 404
    return jsonify(libro)
```

Spiegazione:

1. La route accetta un parametro dinamico `<int:libro_id>` , che identifica il libro da cercare.

2. Usiamo una **comprehension con** `next()` per trovare il libro nella lista `libri`. Se non viene trovato, `libro` assume il valore `None`.
3. Controlliamo se `libro` è `None`:
   - Se sì, restituiamo un **messaggio di errore 404** in formato JSON, indicando che la risorsa non esiste.
   - Se no, restituiamo i dati del libro richiesto.

Questo approccio permette di **gestire errori specifici per ogni route**, mantenendo il client sempre informato sullo stato della richiesta.